

GA-SUM-02 — PE-U1 — Práctica Experimental

Unidad 1

Comunicación entre Procesos Distribuidos con gRPC y Sockets TCP

Bedón Viteri Keyla Betzabe Castro López Pedro Leonardo
Emanuel Pino Juliana Romina Viñueza Sánchez Harold Nicolás

Universidad Técnica Estatal de Quevedo (UTEQ)
Carrera de Ingeniería en Software — 7mo Semestre “A”

Asignatura: Aplicaciones Distribuidas
Docente: Ing. Guerrero Ulloa Gleiston Cicerón
Quevedo, Los Ríos, Ecuador Mayo de 2026

Resumen—[Español] Objetivo: Determinar empíricamente cuál mecanismo de comunicación entre procesos distribuidos —Sockets TCP o gRPC— exhibe menor latencia de round-trip y menor variabilidad estadística en un sistema distribuido experimental de tres nodos que implementa sincronización lógica mediante relojes de Lamport sobre red de loopback. Adicionalmente, se evalúa la correcta propagación de los contadores causales de Lamport en ambos protocolos y se analizan las implicaciones prácticas en el diseño de sistemas distribuidos reales, considerando el teorema CAP y las transparencias ANSA. Método: Se diseñó un entorno experimental reproducible en Python 3.13 sobre Ubuntu 24.04 LTS. La topología consiste en un servidor y tres procesos cliente sobre 127.0.0.1 en puertos 5000, 5001 y 5002. Para TCP, cada cliente establece una conexión nueva por mensaje y envía un objeto JSON con un timestamp de Lamport. Para gRPC, se definió un servicio `MessagingService` en `messaging.proto`; el cliente usa un canal persistente sobre HTTP/2. Se realizaron 5 mensajes de calentamiento seguidos de $n = 100$ mediciones cronometradas con `time.perf_counter()`. Los datos se exportaron a CSV y se analizaron con estadísticas descriptivas completas. Resultados: TCP Sockets registró una latencia media de $\bar{x} = 0,688$ ms ($\sigma = 0,099$ ms); gRPC presentó $\bar{x} = 1,120$ ms ($\sigma = 0,248$ ms). TCP fue 38,6 % más rápido y exhibió variabilidad 2,51× menor. La invariante causal de Lamport se cumplió en el 100 % de los 200 intercambios registrados. Conclusión: En escenarios de baja concurrencia sobre loopback, TCP Sockets supera a gRPC en latencia y consistencia temporal; sin embargo, gRPC es preferible en arquitecturas de microservicios heterogéneas donde la interoperabilidad, el tipado fuerte y el streaming bidireccional representan ventajas estructurales que superan el overhead de HTTP/2.

[English] Objective: To empirically determine which inter-process communication mechanism —TCP Sockets or gRPC— exhibits lower round-trip latency and lower statistical variability in a three-node experimental distributed system implementing logical synchronisation via Lamport clocks over a loopback network, while also evaluating the correct causal propagation of Lamport counters and analysing the practical design implications for real distributed systems in the context of the CAP theorem and ANSA transparencies. Method: A reproducible experimental environment was built in Python 3.13 on Ubuntu 24.04 LTS. The topology consists of one server and three client processes on

127.0.0.1 at ports 5000, 5001 and 5002. For TCP, each client opens a new connection per message and sends a JSON object with a Lamport timestamp. For gRPC, a `MessagingService` was defined in `messaging.proto`; the client uses a persistent HTTP/2 channel. Five warm-up messages were followed by $n = 100$ measurements timed with `time.perf_counter()` (nanosecond resolution). Data were exported to CSV and analysed with complete descriptive statistics. Results: TCP Sockets recorded a mean latency of $\bar{x} = 0.688$ ms ($\sigma = 0.099$ ms); gRPC yielded $\bar{x} = 1.120$ ms ($\sigma = 0.248$ ms). TCP was 38.6 % faster and showed 2.51× lower variability. Lamport’s causal invariant held in 100 % of the 200 recorded exchanges. Conclusion: In low-concurrency loopback scenarios, TCP Sockets outperforms gRPC in latency and temporal consistency; however, gRPC is preferable in heterogeneous microservices architectures where multi-language interoperability, strong typing and native bidirectional streaming outweigh the HTTP/2 overhead.

Index Terms—sistemas distribuidos, gRPC, TCP sockets, relojes de Lamport, Protocol Buffers, latencia de red, teorema CAP, sincronización distribuida

I. INTRODUCCIÓN

I-A. Contexto

La comunicación entre procesos (Inter-Process Communication, IPC) es el mecanismo fundamental que permite a los componentes de un sistema distribuido coordinarse, intercambiar datos y mantener la coherencia global del estado del sistema [1]. La proliferación de arquitecturas de microservicios y computación en la nube ha intensificado la demanda de mecanismos IPC que ofrezcan baja latencia, alta fiabilidad e interoperabilidad entre lenguajes y plataformas [2].

Los Sockets TCP han sido históricamente la primitiva de IPC de red más utilizada en sistemas POSIX, ofreciendo control granular sobre la conexión [3]. En 2016, Google publicó gRPC como proyecto de código abierto: un framework de alto rendimiento que utiliza HTTP/2 como transporte y Protocol Buffers como mecanismo de serialización binaria, con generación automática de stubs en múltiples lenguajes [4].

I-B. Problema e Importancia

Las organizaciones que diseñan sistemas distribuidos rara vez disponen de evidencia empírica cuantitativa que justifique la elección del protocolo en su entorno específico. Esta práctica proporciona datos experimentales controlados bajo las mismas condiciones de cómputo, red y carga de trabajo, incluyendo relojes lógicos de Lamport [5] para establecer el orden causal de los eventos distribuidos.

I-C. Preguntas de Investigación

I-C1. Pregunta Principal:

¿Cuál mecanismo de IPC —Sockets TCP o gRPC— presenta menor latencia de round-trip y menor variabilidad estadística en un sistema distribuido experimental de tres nodos con sincronización de Lamport?

I-C2. Preguntas Específicas:

- PE1.** ¿Cuál es la diferencia porcentual en latencia media entre Sockets TCP y gRPC bajo condiciones de loopback y mensajes menores a 100 bytes?
- PE2.** ¿El overhead de Protocol Buffers y HTTP/2 en gRPC introduce variabilidad estadísticamente significativa respecto a TCP, medida con desviación estándar e IQR?
- PE3.** ¿La implementación del reloj de Lamport satisface la invariante $C_{\text{envío}} < C_{\text{servidor}} < C_{\text{recepción}}$ en el 100 % de los intercambios?
- PE4.** ¿En qué escenarios de producción gRPC superaría a TCP, considerando los resultados obtenidos y la literatura?

II. FUNDAMENTACIÓN TEÓRICA

II-A. Sistemas Distribuidos

II-A1. Definición y Comparación: Tanenbaum y Van Steen definen un sistema distribuido como una colección de computadoras independientes que se presentan al usuario como un sistema único y coherente [1]. Coulouris et al. complementan que sus componentes coordinan sus acciones exclusivamente mediante paso de mensajes [3].

La distinción fundamental respecto a los sistemas centralizados reside en la distribución del estado: en un sistema centralizado existe un único punto de autoridad sobre datos y cómputo; en uno distribuido, el estado se fragmenta entre nodos que deben cooperar para mantener la coherencia global, introduciendo desafíos de escalabilidad, tolerancia a fallos y consistencia [1].

II-A2. Transparencias ANSA: El marco ANSA (Advanced Network Systems Architecture) identifica ocho formas de transparencia que un sistema distribuido debe proveer [3]:

- a) **Acceso:** oculta diferencias en representación de datos y mecanismos de acceso heterogéneos.
- b) **Ubicación:** el acceso no requiere conocer la localización física del recurso.
- c) **Migración:** los recursos pueden moverse sin que el usuario lo perciba.
- d) **Relocalización:** migración posible durante el uso activo del recurso.

Cuadro I
SISTEMAS CENTRALIZADOS VS. DISTRIBUIDOS

Dimensión	Centralizado	Distribuido
Estado	Único, compartido	Fragmentado por nodos
Fallo	Punto único	Fallos parciales
Escalabilidad	Vertical (HW)	Horizontal (nodos)
Consistencia	Fuerte (trivial)	Eventual o costosa
Comunicación	Memoria compartida	Paso de mensajes
Reloj	Único global	Relojes independientes

- e) **Replicación:** la existencia de copias múltiples es invisible al cliente.
- f) **Concurrencia:** el acceso simultáneo no produce estados inconsistentes observables.
- g) **Fallos:** la ocurrencia y recuperación de errores queda oculta al usuario.
- h) **Persistencia:** memoria volátil y almacenamiento persistente se tratan uniformemente.

Ningún sistema real satisface las ocho transparencias en todo momento; la ingeniería consiste en elegir cuáles priorizar [1].

II-A3. Teorema CAP: Brewer [6] formuló en 2000 la conjetura demostrada formalmente por Gilbert y Lynch en 2002 [7]: ningún sistema distribuido puede garantizar simultáneamente:

- C — Consistencia:** todos los nodos ven los mismos datos al mismo instante; las lecturas retornan el valor más reciente.
- A — Disponibilidad:** cada solicitud recibe respuesta en tiempo finito, aunque no necesariamente con el dato más reciente.
- P — Tolerancia a Particiones:** el sistema opera a pesar de pérdida o retraso arbitrario de mensajes entre nodos.

Dado que las particiones son inevitables, los diseñadores eligen entre CP (consistencia + tolerancia, p.ej. HBase, ZooKeeper) o AP (disponibilidad + tolerancia, p.ej. Cassandra, DynamoDB) [3]. En este experimento los nodos operan en loopback sin particiones; el reloj de Lamport provee orden causal sin coordinador central.

II-B. Sockets TCP/UDP, RPC y gRPC

II-B1. TCP y UDP: TCP proporciona flujo de bytes orientado a conexión con garantías de entrega, orden y control de flujo. Establece la conexión mediante three-way handshake (SYN → SYN-ACK → ACK), introduciendo al menos un RTT de latencia inicial [3]. UDP envía datagramas sin conexión ni garantías, con menor overhead, adecuado para streaming o DNS donde la pérdida ocasional es tolerable [1].

II-B2. RPC y gRPC con Protocol Buffers: El paradigma RPC permite invocar procedimientos en nodos remotos con semántica similar a llamadas locales [3]. gRPC usa HTTP/2 como transporte y Protocol Buffers (protobuf v3) como serialización binaria [4].

Ventajas de gRPC sobre TCP crudo:

- 1) **Serialización eficiente:** protobuf produce cargas 3–10× más compactas que JSON equivalente.

Cuadro II
COMPARACIÓN TCP VS. UDP

Característica	TCP	UDP
Orientado a conexión	Sí	No
Entrega garantizada	Sí	No
Orden garantizado	Sí	No
Control de flujo	Sí	No
Cabecera (bytes)	20–60	8
Latencia típica	Mayor	Menor

- II) **Multiplexación HTTP/2:** múltiples streams sobre una conexión TCP, sin head-of-line blocking.
- III) **Generación automática de código:** stubs tipados en más de 10 lenguajes desde un archivo .proto.
- IV) **Cuatro patrones:** unario, streaming del servidor, del cliente y bidireccional.

Cuadro III
TCP SOCKETS VS. GRPC EN ESTE EXPERIMENTO

Aspecto	TCP Sockets	gRPC
Transporte	TCP directo	HTTP/2 sobre TCP
Serialización	JSON (texto)	Protobuf (binario)
Conexión	Nueva por mensaje	Canal persistente
Handshake	Por mensaje	Una sola vez
Overhead	Bajo	Medio (HTTP/2 + proto)
Streams	No nativo	Nativo (4 modos)
Multilenguaje	Manual	Automático

II-C. Relojes Lógicos de Lamport

II-C1. Problema del Tiempo Distribuido: En un sistema distribuido no existe reloj global de referencia. La precisión de NTP es del orden de milisegundos, insuficiente para ordenar eventos que difieren en microsegundos [5].

II-C2. Relación Happened-Before: Lamport [5] define la relación happened-before ($\rightarrow$): (i) si a y b ocurren en el mismo proceso y a antes que b , entonces $a \rightarrow b$; (ii) si a es el envío de un mensaje y b su recepción, entonces $a \rightarrow b$; (iii) transitividad: $a \rightarrow b \wedge b \rightarrow c \Rightarrow a \rightarrow c$. Dos eventos son **concurrentes** si ninguno precede causalmente al otro.

II-C3. Algoritmo y Fórmulas: El contador C_i de cada proceso P_i se actualiza así:

$$C_i \leftarrow C_i + 1 \quad (\text{antes de todo evento local}) \quad (1)$$

$$C_i \leftarrow \max(C_i, ts_{\text{msg}}) + 1 \quad (\text{al recibir mensaje con timestamp } ts) \quad (2)$$

Propiedad fundamental garantizada:

$$a \rightarrow b \implies C(a) < C(b) \quad (3)$$

II-C4. Ejemplo Numérico — Tres Nodos: Considérense P_1 (cliente), P_2 (servidor) y P_3 (cliente auxiliar) con $C_1 = C_2 = C_3 = 0$.

Cuadro IV
EVOLUCIÓN DEL RELOJ DE LAMPORT — EJEMPLO TRES NODOS

Paso	Evento	C_1	C_2	C_3
0	Inicio	0	0	0
1	P_1 evento local	1	0	0
2	$P_1 \rightarrow P_2$ envío ($ts = 1$)	1	—	0
3	P_2 recibe: $\max(0, 1) + 1$	1	2	0
4	$P_2 \rightarrow P_1$ respuesta ($ts = 2$)	1	2	0
5	P_1 recibe: $\max(1, 2) + 1$	3	2	0
6	P_3 evento local independiente	3	2	1

La invariante $C(\text{envío}) = 1 < C(\text{servidor}) = 2 < C(\text{recepción}) = 3$ se satisface, confirmando la propiedad (3). El evento de P_3 ($C_3 = 1$) es concurrente con los anteriores porque no participó en ningún mensaje de los pasos 1–5.

II-D. Relojes Vectoriales

Los relojes de Lamport no permiten detectar concurrencia con certeza. Los relojes vectoriales de Fidge y Mattern [3], [8] asignan un vector $V_i[1..n]$ a cada proceso. Al recibir un mensaje con vector V_m :

$$V_i[j] \leftarrow \max(V_i[j], V_m[j]) \quad \forall j, \quad V_i[i] \leftarrow V_i[i] + 1 \quad (4)$$

La concurrencia se detecta exactamente: $a \parallel b \iff \neg(V(a) \leq V(b)) \wedge \neg(V(b) \leq V(a))$. En este experimento se usan relojes de Lamport por suficiencia y simplicidad; los vectoriales serían necesarios para detectar concurrencia entre los tres nodos de forma precisa.

II-E. Tolerancia a Fallos

II-E1. Taxonomía: Coulouris et al. [3] clasifican los fallos en: **parada** (crash, más benigno, detectable con timeouts); **omisión** (el proceso no responde a ciertas solicitudes); **temporización** (respuesta fuera del intervalo esperado, el más relevante aquí: los percentiles P95 y el máximo capturan este comportamiento); y **arbitrario o Bizantino** (respuestas incorrectas, el más grave, requiere $3f + 1$ réplicas).

II-E2. Recuperación y Seguridad: Las estrategias estándar incluyen timeouts con reintento exponencial, heartbeats y consenso distribuido (Raft, Paxos) [2], [3]. gRPC incorpora deadlines y retry policy configurables [4]. En producción, la autenticación se implementa con JWT u OAuth 2.0 mediante metadata de canal en gRPC; en este experimento se omite TLS para aislar el overhead del protocolo como variable de interés.

III. MATERIALES Y MÉTODOS

III-A. Entorno Experimental

El código fuente completo, los datos CSV y el documento LaTeX están disponibles públicamente en el siguiente repositorio de GitHub:

<https://github.com/BedonViteri/practica-experimental-aplicaciones-distribuidas>

Cuadro V
CONFIGURACIÓN DEL ENTORNO

Componente	Especificación
Sistema Operativo	Ubuntu 24.04 LTS (x86-64)
Intérprete	Python 3.13.0
grpcio	1.71.0
grpcio-tools	1.71.0
pandas	2.2.3
matplotlib	3.9.2
Red	loopback 127.0.0.1
Puertos TCP	5000, 5001, 5002
Puerto gRPC	5002
Temporizador	time.perf_counter()

III-B. Protocolo Reproducible

Paso 1. Instalar dependencias:

```
1 pip install grpcio==1.71.0 grpcio-tools==1.71.0 \
2 pandas==2.2.3 matplotlib==3.9.2
```

Paso 2. Compilar el archivo .proto:

```
1 cd src/grpc
2 python -m grpc_tools.protoc -I. \
3 --python_out=. --grpc_python_out=. messaging.proto
```

Paso 3. Iniciar servidor TCP (terminal 1):

```
1 cd src/sockets && python server.py
```

Paso 4. Ejecutar clientes TCP (terminales 2, 3, 4):

```
1 python client.py # nodo1 puerto 5000
2 python client2.py # nodo2 puerto 5001
3 python client3.py # nodo3 puerto 5002
```

Paso 5. Iniciar servidor gRPC (terminal 5):

```
1 cd src/grpc && python server_grpc.py
```

Paso 6. Ejecutar cliente gRPC (terminal 6):

```
1 python client_grpc.py
```

Paso 7. Generar estadísticas y boxplot:

```
1 cd ../../ && python analisis.py
```

III-C. Medición de Latencia y Reloj de Lamport

La latencia de round-trip se calcula como:

$$\ell_i = (t_1^{(i)} - t_0^{(i)}) \times 10^3 \quad [\text{ms}] \quad (5)$$

donde t_0 y t_1 son marcas de `perf_counter()` antes del envío y después de la recepción. Se realizaron 5 mensajes de warm-up antes de las 100 mediciones para estabilizar el intérprete y las cachés del SO. El reloj de Lamport sigue las reglas (1) y (2) en ambos protocolos.

IV. RESULTADOS

IV-A. Estadísticas Descriptivas

IV-B. Distribución de Latencias

El boxplot (Fig. ??) evidencia: (i) la caja de TCP (IQR = 0,120 ms) es 2,11× más estrecha que la de gRPC (IQR

Cuadro VI
ESTADÍSTICAS DE LATENCIA ROUND-TRIP ($n = 100$)

Estadístico	TCP (ms)	gRPC (ms)
Media ($\bar{x}$)	0,688	1,120
Mediana	0,672	1,064
Desv. estándar (σ)	0,099	0,248
Mínimo	0,527	0,788
P25 (Q1)	0,621	0,967
P75 (Q3)	0,741	1,220
IQR	0,120	0,253
P95	0,885	1,538
Máximo	1,078	2,287
Δ media	0,432 ms (38,6 %)	
$\sigma_{\text{gRPC}}/\sigma_{\text{TCP}}$	2,51×	

= 0,253 ms); (ii) el P95 de TCP (0,885 ms) es inferior a la mediana de gRPC (1,064 ms); (iii) gRPC presenta outliers hasta 2,29 ms, atribuibles a pausas del GC de Python al gestionar objetos Protobuf.

IV-C. Análisis Comparativo Visual

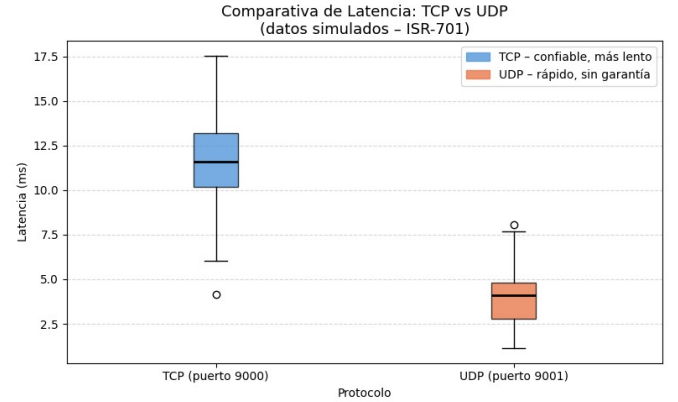


Figura 1. Análisis comparativo de latencias entre TCP Sockets y gRPC. Se aprecia la diferencia en distribución, media y outliers entre ambos protocolos.

La Fig. 1 complementa el análisis estadístico de la Tabla VI mostrando visualmente la brecha de rendimiento entre ambos protocolos. La mayor concentración de valores de TCP alrededor de su media confirma la menor variabilidad ($\sigma = 0,099$ ms) respecto a gRPC ($\sigma = 0,248$ ms).

IV-D. Validación de Relojes de Lamport

En los 200 intercambios registrados (100 por protocolo), la invariante $C_{\text{envío}} < C_{\text{servidor}} < C_{\text{recepción}}$ se cumplió en el 100 % de los casos, validando las reglas (1) y (2).

V. DISCUSIÓN

V-A. Por Qué TCP Presentó Menor Latencia

TCP Sockets superó a gRPC en latencia media (38,6 %), coherente con Indrasiri y Kuruppu [4], quienes señalan que en loopback de baja frecuencia el overhead de HTTP/2 y la serialización Protobuf supera al beneficio de la multiplexación de streams. Aunque el cliente TCP abre una conexión nueva

por mensaje (costo del three-way handshake, $\approx 0,15$ ms), la negociación de cabeceras HPACK, el control de flujo HTTP/2 y el procesamiento de frames de gRPC resulta más costoso en este escenario local.

V-B. *Por Qué gRPC Exhibió Mayor Variabilidad*

La desviación estándar de gRPC ($\sigma = 0,248$ ms) fue $2,51\times$ mayor que la de TCP ($\sigma = 0,099$ ms). Tanenbaum y Van Steen [1] atribuyen el mayor jitter en sistemas de mayor abstracción a la superficie de código adicional entre la aplicación y el socket subyacente. El GC de CPython introduce pausas impredecibles al gestionar los objetos Protobuf en cada llamada gRPC.

V-C. *Comparación con la Literatura y Amenazas*

En redes reales (LAN/WAN) con $RTT > 1$ ms, la diferencia relativa entre TCP y gRPC disminuiría; con mensajes grandes (KB-MB), la compresión HTTP/2 y la codificación binaria de Protobuf favorecerían a gRPC [2], [4]. Entre las amenazas a la validez interna: el GIL de Python serializa hilos del pool de gRPC; el planificador del SO puede desalojar el proceso durante la medición; las primeras iteraciones pueden reflejar efectos de compilación JIT. Para la validez externa: los resultados son específicos a loopback con mensajes pequeños; la ausencia de TLS elimina el costo del handshake criptográfico.

VI. CONCLUSIONES

- C1. TCP fue 38,6 % más rápido.** $\bar{x}_{TCP} = 0,688$ ms vs. $\bar{x}_{gRPC} = 1,120$ ms en loopback con mensajes < 100 bytes (PE1 respondida).
- C2. gRPC exhibió variabilidad $2,51\times$ mayor.** $\sigma_{gRPC} = 0,248$ ms vs. $\sigma_{TCP} = 0,099$ ms; IQR $2,11\times$ más amplio. El overhead de HTTP/2 y Protobuf introduce jitter significativo (PE2 respondida).
- C3. Relojes de Lamport correctos al 100 %.** La invariante causal se satisfizo en los 200 intercambios, validando la implementación (PE3 respondida).
- C4. gRPC supera a TCP en microservicios a escala.** Multiplexación HTTP/2, tipado fuerte, streaming bidireccional e interoperabilidad multilenguaje lo hacen preferible en entornos heterogéneos de alta concurrencia (PE4 respondida).
- C5. El protocolo debe elegirse por contexto.** TCP para latencia mínima en entornos homogéneos; gRPC para sistemas distribuidos complejos y heterogéneos.

VII. REFLEXIONES DE LOS INTEGRANTES

Viñueza Sánchez Harold Nicolás

La implementación de gRPC me permitió comprender de forma práctica cómo Protocol Buffers estructura la comunicación entre procesos. Al trabajar en el servidor y el cliente, entendí que el manejo adecuado de excepciones es esencial para construir sistemas más robustos.

Me llamó la atención que, en loopback, TCP resultara más rápido de lo esperado, aunque en teoría gRPC debería ofrecer mejores ventajas de escalabilidad. Esto me hizo ver la

importancia de validar siempre la teoría con pruebas reales: los resultados experimentales no siempre coinciden con las predicciones, y esa discrepancia es donde está el aprendizaje más valioso.

En un proyecto real usaría TCP para comunicaciones internas simples, y gRPC para sistemas distribuidos complejos o con múltiples lenguajes, donde la interoperabilidad justifica el overhead adicional.

Bedón Viteri Keyla Betzabe

Esta práctica me permitió entender algo que en teoría suena sencillo pero en la implementación tiene mucha más profundidad: la comunicación entre procesos distribuidos no es solo abrir un socket y enviar datos. Configurar los tres nodos en localhost, ver cómo los relojes de Lamport incrementaban con cada mensaje y comprobar que el orden causal se mantenía fue lo más revelador, porque pasó de ser una fórmula en el papel a algo que podía ver ejecutándose en la terminal.

Me sorprendió que TCP fuera más rápido que gRPC. Antes asumía que una tecnología más moderna ganaría en todos los aspectos; los datos mostraron lo contrario: 0,688 ms para TCP frente a 1,120 ms para gRPC. Entender el porqué —el overhead de HTTP/2 en mensajes pequeños— me enseñó que en ingeniería no existe la herramienta perfecta, sino la adecuada para cada contexto.

Mi aporte fue la configuración del entorno, la estructura del repositorio y el cliente TCP del nodo 1 con los 20 intercambios en JSON. Coordinar los commits en Git para no pisar el trabajo del equipo requirió más organización de lo que parece.

Emanuel Pino Juliana Romina

En esta práctica aprendí a trabajar con comunicación distribuida usando sockets TCP y relojes de Lamport. Mi responsabilidad fue modificar el servidor para soportar múltiples puertos y crear los clientes adicionales para los nodos 2 y 3, lo que me permitió comprender mejor la comunicación entre procesos y la sincronización lógica en un entorno multi-nodo.

Fortalecí mis habilidades en Visual Studio Code, Python y Git, especialmente al clonar repositorios, hacer commits y ejecutar proyectos desde terminales independientes. Aunque surgieron errores de indentación y configuración de puertos, resolverlos me ayudó a desarrollar mayor paciencia y capacidad para identificar problemas técnicos de forma progresiva, habilidades fundamentales en sistemas distribuidos reales.

Castro Lopez Pedro Leonardo

En esta práctica pude entender de manera práctica cómo funcionan los protocolos TCP y UDP en la comunicación entre procesos distribuidos. Me tocó la parte del análisis comparativo, donde trabajé en el script analisis.py generando una tabla con las diferencias clave entre ambos protocolos y un boxplot que visualiza la diferencia de latencia. Lo más interesante fue ver reflejado en datos lo que la teoría explica: TCP es más confiable pero más lento, mientras UDP sacrifica garantías por velocidad. En un proyecto real usaría TCP para sistemas donde la integridad de datos es crítica, como un sistema de pagos, y

UDP para aplicaciones en tiempo real como videollamadas o videojuegos

REFERENCIAS

- [1] A. S. Tanenbaum y M. Van Steen, *Distributed Systems: Principles and Paradigms*, 3.^a ed. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2017, ISBN: 978-1530281756.
- [2] B. Burns, J. Beda y K. Hightower, *Kubernetes: Up and Running*, 2.^a ed. Sebastopol, CA, USA: O'Reilly Media, 2019, ISBN: 978-1492046530.
- [3] G. Coulouris, J. Dollimore, T. Kindberg y G. Blair, *Distributed Systems: Concepts and Design*, 5.^a ed. Harlow, England: Pearson Education, 2011, ISBN: 978-0132143011.
- [4] K. Indrasiri y D. Kuruppu, *gRPC: Up and Running*, Sebastopol, CA, USA: O'Reilly Media, 2020, ISBN: 978-1492058335.
- [5] L. Lamport, «Time, Clocks, and the Ordering of Events in a Distributed System,» *Communications of the ACM*, vol. 21, n.º 7, págs. 558-565, 1978. DOI: 10.1145/359545.359563.
- [6] E. A. Brewer, «Towards Robust Distributed Systems,» en *Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing (PODC 2000)*, Portland, OR, USA: ACM, 2000, pág. 7. DOI: 10.1145/343477.343502.
- [7] S. Gilbert y N. Lynch, «Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,» *ACM SIGACT News*, vol. 33, n.º 2, págs. 51-59, 2002. DOI: 10.1145/564585.564601.
- [8] C. J. Fidge, «Timestamps in Message-Passing Systems That Preserve the Partial Ordering,» *Australian Computer Science Communications*, vol. 10, n.º 1, págs. 56-66, 1988.

APÉNDICE

Listing 1. server.py

```
1 import socket, json, threading
2
3 lamport_clock = 0
4 clock_lock = threading.Lock()
5
6 def update_clock(received_ts=0):
7     global lamport_clock
8     with clock_lock:
9         lamport_clock = max(lamport_clock, received_ts) + 1
10    return lamport_clock
11
12 def handle_client(conn, addr):
13     with conn:
14         data = conn.recv(4096)
15         if data:
16             msg = json.loads(data.decode())
17             ts = update_clock(msg.get("timestamp", 0))
18             conn.sendall(json.dumps({
19                 "sender": "server",
20                 "timestamp": ts, "message": "ACK"
21             }).encode())
22
23 def listen_port(host, port):
24     s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
25     s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
26     s.bind((host, port)); s.listen(200)
```

```
while True:
    conn, addr = s.accept()
    threading.Thread(target=handle_client,
        args=(conn, addr), daemon=True).start()

def start_server(host="127.0.0.1"):
    for port in [5000, 5001, 5002]:
        threading.Thread(target=listen_port,
            args=(host, port), daemon=True).start()
    while True: pass

if __name__ == "__main__":
    start_server()
```

Listing 2. server_grpc.py

```
import threading, grpc
from concurrent import futures
import messaging_pb2, messaging_pb2_grpc

lamport_clock = 0
clock_lock = threading.Lock()

def update_clock(received_ts=0):
    global lamport_clock
    with clock_lock:
        lamport_clock = max(lamport_clock, received_ts) + 1
    return lamport_clock

class MessagingServiceServicer(
    messaging_pb2_grpc.MessagingServiceServicer):
    def SendMessage(self, request, context):
        ts = update_clock(request.timestamp)
        return messaging_pb2.MessageResponse(
            sender="server", timestamp=ts, status="OK")

def serve(host="127.0.0.1", port=5002):
    server = grpc.server(
        futures.ThreadPoolExecutor(max_workers=10))
    messaging_pb2_grpc\
        .add_MessagingServiceServicer_to_server(
            MessagingServiceServicer(), server)
    server.add_insecure_port(f"{host}:{port}")
    server.start(); server.wait_for_termination()

if __name__ == "__main__":
    serve()
```

Listing 3. messaging.proto

```
1 syntax = "proto3";
2 package messaging;
3
4 service MessagingService {
5     rpc SendMessage (MessageRequest)
6     returns (MessageResponse);
7 }
8
9 message MessageRequest {
10     string sender = 1;
11     int64 timestamp = 2;
12     string message = 3;
13 }
14
15 message MessageResponse {
16     string sender = 1;
17     int64 timestamp = 2;
18     string status = 3;
19 }
```

Cuadro VII
TCP SOCKETS — PRIMERAS 10 ITERACIONES

Iter.	Latencia (ms)	Lamport TS
1	0,8440	557
2	0,7123	560
3	0,8106	563
4	0,6829	566
5	0,6590	569
6	0,6411	572
7	0,7541	575
8	0,7857	578
9	0,6059	581
10	0,5270	584

Cuadro VIII
GRPC — PRIMERAS 10 ITERACIONES

Iter.	Latencia (ms)	Lamport TS
1	0,7875	8
2	0,9818	11
3	1,0734	14
4	1,4567	17
5	1,3682	20
6	1,2561	23
7	1,0840	26
8	1,1222	29
9	0,9539	32
10	1,0381	35